```
#include <systemc.h>
SC_MODULE(PriorityInterruptController)
{
    sc_event interrupt_event;
    bool irq[3];
    int priority[3];

    SC_CTOR(PriorityInterruptController)
    {
        // Priority: 0 = highest, 2 = lowest
        priority[0] = 0;
        priority[1] = 1;
        priority[2] = 2;

        irq[0] = false;
        irq[1] = false;
        irq[2] = false;

        SC_THREAD(interrupt_handler);
    }

    // Generate an interrupt
    void generate_interrupt(int device)
    {
        irq[device] = true;

        cout << sc_time_stamp()
             << " Device " << device
             << " generated interrupt" << endl;

        interrupt_event.notify();
    }

    // Interrupt handler
    void interrupt_handler()
    {
        while (true)
        {
            wait(interrupt_event);

            int highest_priority = -1;

            // Find highest-priority pending interrupt
            for (int i = 0; i < 3; i++)
            {
                if (irq[i])
                {
                    if (highest_priority == -1 ||
                        priority[i] < priority[highest_priority])
                    {
                        highest_priority = i;
                    }
                }
            }

            if (highest_priority != -1)
            {
                cout << sc_time_stamp()
                     << " Servicing Device "
                     << highest_priority
                     << " (Priority "
                     << priority[highest_priority]
                     << ")" << endl;

                irq[highest_priority] = false;

                // Interrupt service time
                wait(2, SC_NS);

                cout << sc_time_stamp()
```

```
                cout << sc_time_stamp()
                     << " Servicing Device "
                     << highest_priority
                     << " (Priority "
                     << priority[highest_priority]
                     << ")" << endl;

                irq[highest_priority] = false;

                // Interrupt service time
                wait(2, SC_NS);

                cout << sc_time_stamp()
                     << " Device "
                     << highest_priority
                     << " interrupt completed"
                     << endl;
            }
        }
    }
};


// Testbench
SC_MODULE(Testbench)
{
    PriorityInterruptController* controller;

    SC_CTOR(Testbench)
    {
        SC_THREAD(generate_interrupts);
    }

    void generate_interrupts()
    {
        wait(5, SC_NS);

        controller->generate_interrupt(2);

        wait(1, SC_NS);

        controller->generate_interrupt(1);

        wait(1, SC_NS);

        controller->generate_interrupt(0);

        wait(10, SC_NS);

        sc_stop();
    }
};


int sc_main(int argc, char* argv[])
{
    PriorityInterruptController controller(
        "PriorityInterruptController"
    );

    Testbench testbench("Testbench");

    testbench.controller = &controller;

    sc_start();

    return 0;
}
```